Lecture 10 - DMA

Contents

1. Task: Minimal UART with DMA 2
   1.1. Inquiries 2
2. Task: ADC With DMA 3
   2.1. Implementation 4

1. Task: Minimal UART with DMA

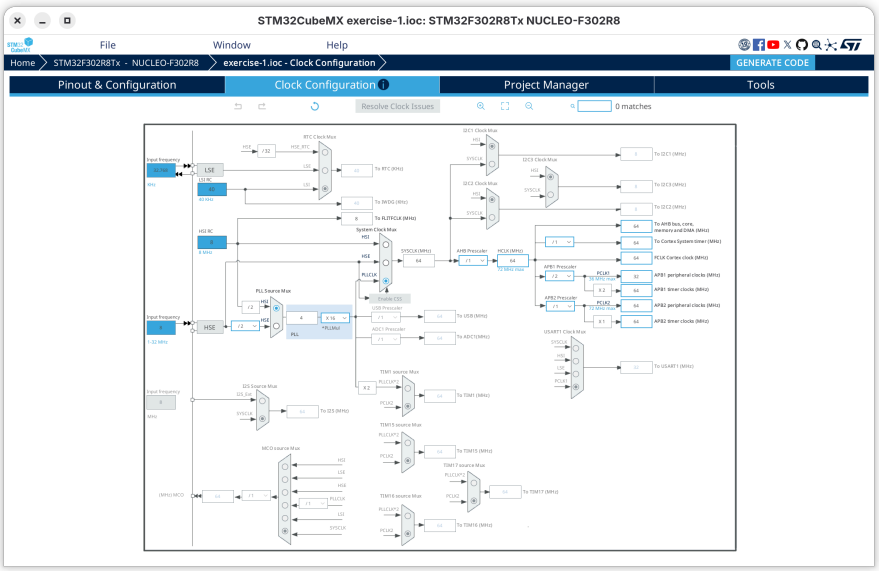


Figure 1: Clock setup for the exercise 1.

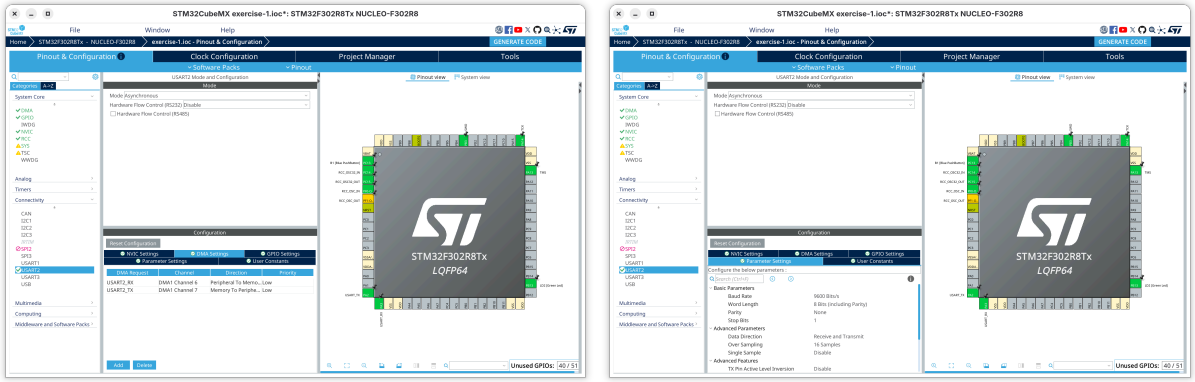


Figure 2: (Left) DMA setup for UART 2. (Right) Basic parameters for UART setup.

1.1. Inquiries

The code `huart.Instance->CR3` is a direct manipulation of the DMA UART manipulation. The operation `|= USART_CR3_DMAT` enables the operation of hardware data transmission. The other option `&= ~USART_CR3_DMAT`, clears the register or at least the DMA enable bit.

It is possible to replicate, or rather opacate the register manipuation of the previous instructions.

Please, refer to the attached videos for the demonstration.

2. Task: ADC With DMA

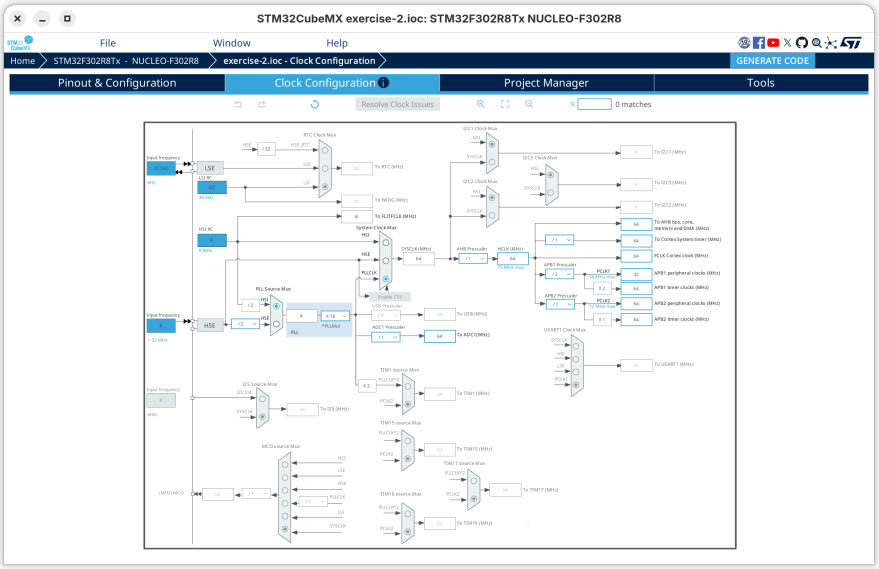


Figure 3: Clock setup for the exercise 2.

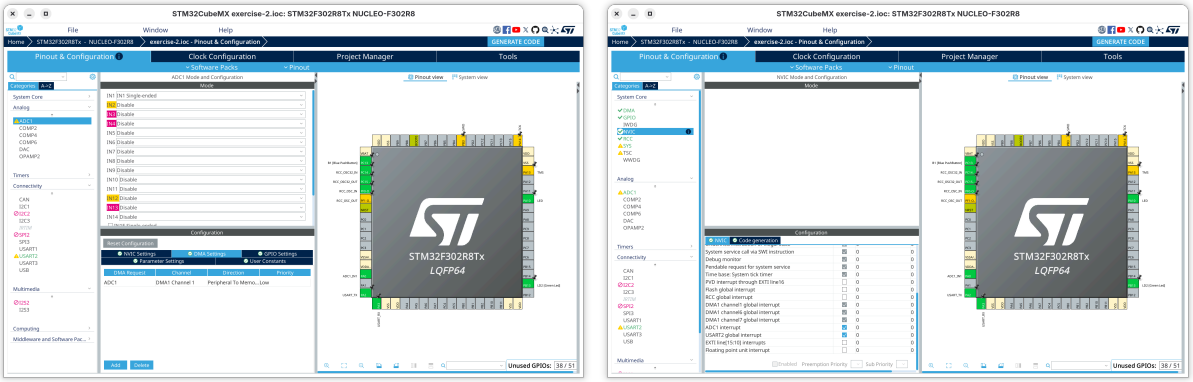


Figure 4: (Left) ADC DMA setup . (Right) NVIC setup.

2.1. Implementation

```
// Start the ADC in DMA mode.
// Because "Continuous Conversion" is enabled, the DMA will continuously
// overwrite adc_ram_buffer[0] in the background automatically.
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_ram_buffer, 1);

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  HAL_Delay(1000);

  float voltage = (adc_ram_buffer[0] / 4095.0f) * 3.3f;
  float temp_celsius = (voltage - 0.5f) * 100.0f;

  sprintf(uart_tx_buffer, "Temp: %.2f C\r\n", temp_celsius);

  HAL_UART_Transmit_DMA(&huart2, (uint8_t*)uart_tx_buffer, strlen(uart_tx_buffer));

  HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);

  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

The implementation uses the `HAL_UART_Transmit_DMA` function that replaces the previous register manipulation.

Please, refer to the attached videos for the demonstration.